C++ Programming Problem Set

Module: Default Arguments (Flexibility, Rules & Memory Mechanics)

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Default Arguments & Function Prototypes

Integrates: I/O, Operators, Loops, Functions, References, Arrays & Strings

Instructions & Pedagogical Guidelines

This problem set evaluates your mastery of C++ default arguments. You will solve programming challenges testing the right-to-left trailing order constraint, declaration vs definition prototype rules, function overloading ambiguity traps, pass-by-reference output parameters with defaults, array transformations, string box formatting, and financial calculation engines.

Level 1: Default Argument Fundamentals & Prototype Mechanics

Problem 1: Flexible Power & Compound Interest Calculator

Default Values | Mathematical Operators | Prototypes

Declare a function `double computeGrowth(double principal, double rate = 5.0, int years = 1)` with default rate (5.0%) and duration (1 year). Prototype the function above `main()` and define it below `main()`. In `main()`, demonstrate three distinct invocations: passing 1 argument, 2 arguments, and all 3 arguments. Display the returned compound amounts using basic I/O formatting.

Default Argument Prototype | Right-to-Left Trailing Rule | Call-Site Substitution

Call Site Example	Evaluated Formula & Output
`computeGrowth(1000.0)`	`$1000.00 * (1 + 0.05)^1 = $1050.00 (Uses rate=5.0, years=1)`
`computeGrowth(1000.0, 8.0, 3)`	`$1000.00 * (1 + 0.08)^3 = $1259.71 (Overrides both defaults)`

Problem 2: Syntax Trap Inspector (Right-to-Left Rule & Redefinition)

Compiler Rules | Error Analysis | Prototypes

Write a program demonstrating the two cardinal rules of default arguments:
1. *Right-to-Left Rule:* Explain why `void setBox(int w = 10, int h, int d = 5)` triggers a compilation error.
2. *Declaration vs Definition Rule:* Write a valid prototype `int add(int a, int b = 10);` above `main()`. Show that attempting to repeat `int add(int a, int b = 10)` in the definition header causes a redefinition compile error.

Cardinal Rule #1 (Trailing Order) | Cardinal Rule #2 (No Redefinition) | Compiler Error Diagnostics

Flawed Code Fragment	Compiler Diagnostic Message
`void func(int x = 5, int y);`	`ERROR: Default argument missing for parameter 2 ('y')!`

Problem 3: Overloading Ambiguity Trap Visualizer

Function Overloading | Ambiguity | Compiler Resolution

Write two overloaded functions:
- `void printMessage(std::string msg)`
- `void printMessage(std::string msg, int repeatCount = 1)`

In `main()`, demonstrate that calling `printMessage("Hello")` results in a compile-time ambiguity error because both functions match the call signature. Fix the ambiguity by removing one overload and relying entirely on default arguments.

Level 2: Intermediate Applications (References, Arrays & Strings)

Problem 4: Array Multi-Scaler with Reference Output Tracker

Arrays | Pass-by-Reference | Default Scale Factor

Implement a function `void scaleArray(int arr[], int size, int &modCount, int factor = 2, bool doubleOdds = false)`. The function multiplies every array element by `factor`. If `doubleOdds` is set to `true`, odd elements receive an extra 2x multiplier. Update `modCount` via pass-by-reference to track how many elements were modified. Call the function in `main()` with 2, 3, and 4 arguments.

```
// Scaler Prototype with default trailing factor and flag: void scaleArray(int arr[], int size, int &modCount, int factor = 2, bool doubleOdds = false);
```

Input Array & Parameters	Updated Array & Reference Tracker
`[1, 2, 3, 4], factor=3, doubleOdds=true`	`Array: [6, 6, 18, 12] | Modified Count: 4`

Problem 5: In-Place Text Box Frame Generator

Strings | Default Fill Characters | Loops

Write a function `void printBoxedText(const std::string &text, char borderChar = '*', int padding = 2, bool center = true)`. The function prints the provided text enclosed inside a drawn border box. Test the function in `main()` using custom text strings with default borders, overridden custom border characters (e.g., `'#'`), and custom padding.

Const Reference String | Default Border Fill | Console Alignment Loops

Call Site	Rendered Console Frame

`printBoxedText("C++", '#', 1)`	`#####`
	`# C++ #`
	`#####`

Problem 6: Sentinel Array Searcher with Default Step & Direction

Array Searching | Default Step | Direction Flags

Write a search function `int findElement(const int arr[], int size, int target, int step = 1, bool searchForward = true)`. If `searchForward` is true, search from index 0 upward by `step` increments; otherwise search from index `size-1` downward. Return the found index or `-1` if missing. Test searching an array using default step 1, custom step 2, and reverse search.

Default Step Offset | Directional Searching | Const Array Protection

Array Input	Search Output
`[10, 20, 30, 40, 50], target=40, step=1, forward=false`	`Found target 40 at Index 3 (Reverse Search)`

Level 3: Inline Integration, Pointers & Comprehensive Capstone

Problem 7: Inline Utility Engine with Multi-Tier Defaults

Inline Functions | Default Arguments | Speed

Combine inline functions with default arguments. Implement an inline function `inline double calculateTax(double grossAmount, double taxRate = 18.0, double discount = 0.0)`. Demonstrate that combining `inline` in-place code expansion with default parameter substitution eliminates runtime jump overhead while offering simple API ergonomics. Process an array of 5 invoice amounts inside a benchmark loop.

```
// Combined Inline Keyword and Default Parameters: inline double calculateTax(double grossAmount, double taxRate = 18.0, double discount = 0.0) { double taxable = grossAmount - discount; return (taxable > 0) ? (taxable * taxRate / 100.0) : 0.0; }
```

Invoice Amount	Calculated Output
$500.00 (Using default 18% tax and $0 discount)	Tax Due: $90.00 (Zero-overhead inline expansion)

Problem 8: Pointer & Reference Output Parameter Integrator

Call-by-Address | Call-by-Reference | Defaults

Design a function `bool processTransaction(double amount, double &balance, double *fee = nullptr, double overdraftLimit = 500.0)`. The function deducts `amount` from `balance` if `(balance + overdraftLimit) >= amount`. If pointer `fee` is not `nullptr`, apply a transaction fee stored at `*fee`; otherwise apply a default $2.00 fee. Return `true` if transaction succeeded, or `false` if overdraft limit was exceeded.

Nullptr Pointer Defaults | Reference Balance Mutation | Overdraft Guard Logic

Initial Balance & Call	Updated Ledger Result
Balance: $100.00, Withdraw: $150.00 (fee=nullptr)	Status: SUCCESS | New Balance: -$52.00 ($2 default fee applied)

Problem 9: Cumulative Capstone: Flexible Employee Salary Ledger

Comprehensive Integration | Structures | Default Parameters

Build a payroll system. Define a structure `Employee` with `id`, `name`, and `baseSalary`. Implement functions:
1. `double computeNetSalary(const Employee &emp, double bonus = 0.0, double taxRate = 15.0, double healthDeduction = 50.0)`
2. `void generatePayStub(const Employee &emp, double bonus = 0.0, double taxRate = 15.0, double healthDeduction = 50.0)`
In `main()`, process an array of 4 employees, calling the functions with varying argument counts (default bonus/tax vs customized rates).

Struct Const References | Multi-Parameter Defaults | Complete Module Capstone

Employee Record	Pay Stub Output
ID: 101, Name: Gaurav, Base: $4,000	Bonus: $0 | Tax: 15% | Net Salary: $3,350.00
ID: 102, Name: Alice, Base: $5,000	Bonus: $500 | Tax: 20% | Net Salary: $4,350.00 (Customized)